

Национальный исследовательский университет «Высшая школа экономики»
Московский институт электроники и математики им. А.Н. Тихонова

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1

по дисциплине «Базы данных»

Тема: Проектирование базы данных
Модель «Филателистическое агентство»
ВАРИАНТ 14

Выполнил: студент группы БПМ232
Сулимов Игорь Андреевич

Москва, 2026

Содержание

1 Системный анализ предметной области	3
1.1 Сущности	3
1.2 Связи между сущностями	3
2 Логическая модель БД (ER-диаграмма)	4
3 Запросы на языке реляционной алгебры	4
3.1 Запрос 1: Список всех УФПС, которым необходим заданный вид ФП	4
3.2 Запрос 2: Список УФПС, которые не оставляли заявки за последние три месяца	5
3.3 Запрос 3: УФПС, заказавшие те же ФП, что и заданное УФПС в указанный день. Вывести УФПС – ФП – Количество в заказе.	5
3.4 Запрос 4: Список ФП, которые указанное УФПС заказало больше двух раз	6
3.5 Запрос 5: УФПС, которым требуется не больше заданного количества заданной ФП. Вывести Номер заявки – УФПС – Количество – Дата заявки.	6
3.6 Запрос 6: Получить тройки УФПС – Вид ФП – Количество ФП, расположенных в заданном регионе по заявкам последней недели. Учесть следующее: если от одного УФПС поступило несколько заявок на ФП одного вида, следует выводить суммарное количество	7
3.7 Запрос 7: Получить список из двух (самый популярный и самый непопулярный) видов ФП. Вывести Вид ФП – Количество заявок на него – Дата последней заявки.	7
3.8 Запрос 8: Получить список из трёх самых популярных ФП указанного вида за указанный месяц (т.е. те ФП, которые заказали больше других). Вывести ФП – Процент от общего количества ФП того же вида.	8
4 Группы пользователей и ожидаемые запросы	8
4.1 Группа 1: Менеджеры Филателистического агентства	8
4.2 Группа 2: Руководители УФПС	9
4.3 Группа 3: Региональные аналитики	9
4.4 Группа 4: Операторы обработки заявок	9
5 Нормальные формы	9
5.1 Функциональные зависимости (основные)	9
5.2 Проверка 1НФ (Первая нормальная форма)	9
5.3 Проверка 2НФ (Вторая нормальная форма)	10
5.4 Проверка 3НФ (Третья нормальная форма)	10
5.5 Проверка BCNF (Нормальная форма Бойса–Кодда)	11
5.5.1 Детальный анализ таблицы ufps	11

6	Реализация БД на SQL	11
6.1	Скрипт создания схемы и таблиц	11
6.2	Скрипт заполнения (INSERT)	12

1 Системный анализ предметной области

Предметная область «Филателистическое агентство» включает учет видов филателистической продукции (ФП), конкретных позиций ФП, управлений федеральной почтовой связи (УФПС), регионов РФ и заявок на поставку ФП от УФПС.

1.1 Сущности

- **fp_types** — справочник видов ФП (марки, открытки, конверты и т.п.).
- **regions** — справочник регионов РФ.
- **periods** — справочник периодов (неделя, месяц, квартал, год).
- **ufps** — управления федеральной почтовой связи.
- **philatelic_products** — конкретная филателистическая продукция.
- **requests** — заявки от УФПС.
- **request_items** — состав заявок (позиции ФП и количество).

1.2 Связи между сущностями

1. **regions 1:N ufps** — каждый УФПС расположен в одном регионе, регион содержит много УФПС.
2. **fp_types 1:N philatelic_products** — каждая позиция ФП принадлежит одному виду.
3. **ufps 1:N requests** — каждый УФПС может формировать много заявок.
4. **periods 1:N requests** — у каждой заявки может быть один период.
5. **requests 1:N request_items** — заявка может содержать несколько строк товаров.
6. **philatelic_products 1:N request_items** — одна позиция ФП может встречаться в разных заявках.
7. **requests N:M philatelic_products** — реализована через промежуточную таблицу; одна заявка может содержать много товаров, один товар может встречаться во многих заявках **request_items**.

1. Выполняется соединение $U \bowtie Q \bowtie I \bowtie P \bowtie T$ для получения УФПС, заявок, позиций и вида ФП.
2. Операция селекции $\sigma_{T.type_name=':type'}$ оставляет только строки с заданным видом ФП.
3. Операция проекции $\pi_{U.ufps_name}$ выводит только названия УФПС.

3.2 Запрос 2: Список УФПС, которые не оставляли заявки за последние три месяца

Запрос:

$$\pi_{ufps_id,ufps_name}(U) - \pi_{U.ufps_id,U.ufps_name} (U \bowtie_{U.ufps_id=Q.ufps_id} \sigma_{Q.request_date \geq \text{CURRENT_DATE}-90}(Q))$$

Пояснение:

1. Первая часть $\pi_{ufps_id,ufps_name}(U)$ — множество всех УФПС.
2. Во второй части выбираются УФПС, у которых есть заявки за последние 90 дней.
3. Разность множеств — возвращает УФПС, не имеющие заявок в этот период.

3.3 Запрос 3: УФПС, заказавшие те же ФП, что и заданное УФПС в указанный день. Вывести УФПС – ФП – Количество в заказе.

Запрос:

Переименования (алиасы) для самосоединения:

$$Q1 = \rho_{Q1}(Q), \quad Q2 = \rho_{Q2}(Q), \quad I1 = \rho_{I1}(I), \quad I2 = \rho_{I2}(I), \quad U2 = \rho_{U2}(U)$$

где ρ — операция переименования отношения (alias), то есть логическая “копия” таблицы с новым именем в рамках одного запроса.

$$\begin{aligned} & \pi_{U2.ufps_name, P.product_name, I2.quantity} \left(\right. \\ & \quad \sigma_{\substack{Q1.ufps_id=:ufps \wedge \\ Q1.request_date=:d \wedge \\ Q2.request_date=:d \wedge \\ U2.ufps_id \neq :ufps}} (Q1 \bowtie_{Q1.request_id=I1.request_id} I1 \\ & \quad \bowtie_{I1.product_id=I2.product_id} I2 \\ & \quad \bowtie_{I2.request_id=Q2.request_id} Q2 \\ & \quad \bowtie_{Q2.ufps_id=U2.ufps_id} U2 \\ & \quad \left. \bowtie_{I2.product_id=P.product_id} P \right) \end{aligned}$$

Пояснение:

1. Формируются две «копии» заявок: $Q1/I1$ (заданное УФПС) и $Q2/I2$ (другие УФПС).
2. Соединение по $I1.product_id = I2.product_id$ оставляет одинаковые позиции ФП.
3. Фильтр по дате $Q1.request_date =: d$ и $Q2.request_date =: d$ ограничивает выбор указанным днем.
4. Условие $U2.ufps_id \neq: ufps$ исключает исходное УФПС.
5. Проекция выводит: УФПС — ФП — Количество.

3.4 Запрос 4: Список ФП, которые указанное УФПС заказало больше двух раз

Запрос:

$$\pi_{product_name} \left(\sigma_{cnt > 2} \left(\gamma_{P.product_id, P.product_name; COUNT(*) \rightarrow cnt} \left(\sigma_{Q.ufps_id=:ufps} \left(Q \bowtie_{Q.request_id=I.request_id} I \right. \right. \right. \right. \\ \left. \left. \left. \left. \bowtie_{I.product_id=P.product_id} P \right) \right) \right) \right)$$

Пояснение:

1. Выбираются заявки только указанного УФПС: $\sigma_{Q.ufps_id=:ufps}$.
2. По каждой ФП считается число вхождений в заявках ($COUNT(*)$).
3. Отбираются позиции, где $cnt > 2$.
4. Выводятся наименования ФП.

3.5 Запрос 5: УФПС, которым требуется не больше заданного количества заданной ФП. Вывести Номер заявки – УФПС – Количество – Дата заявки.

Запрос:

$$\pi_{Q.request_id, U.ufps_name, I.quantity, Q.request_date} \left(\sigma_{P.product_name=:product' \wedge (U \bowtie_{U.ufps_id=Q.ufps_id} Q \right. \\ \left. I.quantity \leq: maxq \right. \\ \left. \bowtie_{Q.request_id=I.request_id} I \right. \\ \left. \bowtie_{I.product_id=P.product_id} P) \right)$$

Пояснение:

1. Соединяются УФПС, заявки, строки заявок и продукция.
2. Фильтруются строки по заданной ФП и условию $quantity \leq: maxq$.
3. В результате выводятся: номер заявки, УФПС, количество, дата заявки.

3.6 Запрос 6: Получить тройки УФПС – Вид ФП – Количество ФП, расположенных в заданном регионе по заявкам последней недели. Учесть следующее: если от одного УФПС поступило несколько заявок на ФП одного вида, следует выводить суммарное количество

Запрос:

$$\gamma_{U.ufps_name, T.type_name; SUM(I.quantity) \rightarrow total_qty} \left(\sigma_{\begin{array}{l} R.region_name='region' \wedge \\ Q.request_date \geq CURRENT_DATE-7 \end{array}} \left(R \bowtie_{R.region_id=U.region_id} U \right. \right. \\ \left. \bowtie_{U.ufps_id=Q.ufps_id} Q \right. \\ \left. \bowtie_{Q.request_id=I.request_id} I \right. \\ \left. \bowtie_{I.product_id=P.product_id} P \right. \\ \left. \bowtie_{P.fp_type_id=T.fp_type_id} T \right)$$

Пояснение:

1. Соединяются регион, УФПС, заявки, строки заявок, продукция и виды ФП.
2. Отбор выполняется по заданному региону и последним 7 дням.
3. Группировка по паре (УФПС, вид ФП).
4. Для каждой группы считается суммарное количество $SUM(quantity)$.

3.7 Запрос 7: Получить список из двух (самый популярный и самый непопулярный) видов ФП. Вывести Вид ФП – Количество заявок на него – Дата последней заявки.

Запрос:

$$top_{1,\downarrow}(A) \cup top_{1,\uparrow}(A), \\ A = \gamma_{T.type_name; COUNT(I.request_item_id) \rightarrow req_cnt, MAX(Q.request_date) \rightarrow last_date} (T \bowtie_{T.fp_type_id=P.fp_type_id} P \\ \bowtie_{P.product_id=I.product_id} I \\ \bowtie_{I.request_id=Q.request_id} Q)$$

Пояснение:

1. Для каждого вида ФП вычисляется число заявок (строк заявок) и дата последней заявки.
2. $top_{1,\downarrow}(A)$ выбирает самый популярный вид.

4.2 Группа 2: Руководители УФПС

- Какие УФПС заказывают те же позиции ФП, что и наше УФПС?
- Какие наши заявки содержат дефицитные/редкие позиции?

4.3 Группа 3: Региональные аналитики

- Какие УФПС региона не подавали заявок за последние 3 месяца?
- Каково суммарное количество заказов по видам ФП за последнюю неделю по региону?

4.4 Группа 4: Операторы обработки заявок

- Какие заявки на заданную ФП имеют количество не выше порога?
- Какие УФПС запрашивают конкретный вид ФП прямо сейчас?

5 Нормальные формы

5.1 Функциональные зависимости (основные)

- $fp_type_id \rightarrow type_name$
- $region_id \rightarrow region_name$
- $period_id \rightarrow period_name, days_count$
- $ufps_id \rightarrow ufps_name, region_id, address$
- $product_id \rightarrow fp_type_id, product_name, issue_date, nominal_value$
- $request_id \rightarrow ufps_id, request_date, period_id, status$
- $request_item_id \rightarrow request_id, product_id, quantity$

5.2 Проверка 1НФ (Первая нормальная форма)

Условия 1НФ:

- Все атрибуты атомарны;
- Нет повторяющихся групп.

Проверка:

- В таблицах `fp_types`, `regions`, `periods`, `ufps`, `philatelic_products`, `requests`, `request_items` каждое поле хранит одно значение;

- Состав заявки вынесен в отдельную таблицу `request_items`, то есть множественные товары не хранятся в одном поле;
- Каждая таблица имеет первичный ключ (... `_id`), однозначно идентифицирующий запись.

Вывод: база данных соответствует 1НФ.

5.3 Проверка 2НФ (Вторая нормальная форма)

Условия 2НФ:

- Находится в 1НФ;
- Каждый неключевой атрибут полностью зависит от первичного ключа (нет частичных зависимостей).

Проверка:

- Во всех таблицах используются простые первичные ключи (`fp_type_id`, `region_id`, `period_id`, `ufps_id`, `product_id`, `request_id`, `request_item_id`);
- В таблице `request_items` атрибуты `request_id`, `product_id`, `quantity` зависят от полного первичного ключа `request_item_id`;
- Частичные зависимости отсутствуют.

Вывод: база данных соответствует 2НФ.

5.4 Проверка 3НФ (Третья нормальная форма)

Условия 3НФ:

- Находится в 2НФ;
- Отсутствуют транзитивные зависимости (неключевые атрибуты не зависят от других неключевых атрибутов).

Анализ транзитивных зависимостей:

- В таблице `ufps` хранится только ссылка на регион (`region_id`), а название региона вынесено в `regions`;
- В таблице `philatelic_products` хранится ссылка на вид ФП (`fp_type_id`), а название вида вынесено в `fp_types`;
- В таблице `requests` хранится ссылка на УФПС (`ufps_id`) и период (`period_id`), а их описательные атрибуты находятся в отдельных таблицах;
- В таблице `request_items` отсутствуют дублирующие описательные поля продукции или заявки.

Вывод: база данных соответствует 3НФ.

5.5 Проверка BCNF (Нормальная форма Бойса–Кодда)

Условия BCNF:

- Находится в 3НФ;
- Для любой нетривиальной функциональной зависимости $X \rightarrow Y$, X должен быть суперключом.

5.5.1 Детальный анализ таблицы ufps

В таблице ufps рассматриваются зависимости:

1. $\text{ufps_id} \rightarrow \text{ufps_name}, \text{region_id}, \text{address}$ — ufps_id является РК (суперключ);
2. $\text{ufps_name} \rightarrow \text{ufps_id}, \text{region_id}, \text{address}$ — при ограничении UNIQUE на ufps_name это также суперключ.

Аналогично в справочниках (fp_types , regions , periods) левая часть зависимостей является ключом (РК или UNIQUE-ключом), а в операционных таблицах детерминантами выступают первичные ключи.

Вывод: база данных соответствует BCNF.

6 Реализация БД на SQL

6.1 Скрипт создания схемы и таблиц

```
DROP SCHEMA IF EXISTS philately CASCADE;
CREATE SCHEMA philately;
SET search_path TO philately;

CREATE TABLE fp_types (
    fp_type_id SERIAL PRIMARY KEY,
    type_name VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE regions (
    region_id SERIAL PRIMARY KEY,
    region_name VARCHAR(120) NOT NULL UNIQUE
);

CREATE TABLE periods (
    period_id SERIAL PRIMARY KEY,
    period_name VARCHAR(20) NOT NULL UNIQUE,
    days_count INT NOT NULL CHECK (days_count > 0)
```

```

);

CREATE TABLE ufps (
    ufps_id SERIAL PRIMARY KEY,
    ufps_name VARCHAR(150) NOT NULL UNIQUE,
    region_id INT NOT NULL REFERENCES regions(region_id),
    address VARCHAR(255)
);

CREATE TABLE philatelic_products (
    product_id SERIAL PRIMARY KEY,
    fp_type_id INT NOT NULL REFERENCES fp_types(fp_type_id),
    product_name VARCHAR(180) NOT NULL,
    issue_date DATE,
    nominal_value NUMERIC(10,2) CHECK (nominal_value >= 0)
);

CREATE TABLE requests (
    request_id SERIAL PRIMARY KEY,
    ufps_id INT NOT NULL REFERENCES ufps(ufps_id),
    request_date DATE NOT NULL,
    period_id INT REFERENCES periods(period_id),
    status VARCHAR(30) NOT NULL DEFAULT 'new'
);

CREATE TABLE request_items (
    request_item_id SERIAL PRIMARY KEY,
    request_id INT NOT NULL REFERENCES requests(request_id) ON DELETE
        CASCADE,
    product_id INT NOT NULL REFERENCES philatelic_products(product_id),
    quantity INT NOT NULL CHECK (quantity > 0)
);

```

6.2 Скрипт заполнения (INSERT)

```

INSERT INTO fp_types(type_name) VALUES
('Marka'), ('Otkrytka'), ('Konvert'), ('Blok'), ('Kartmaksimum');

INSERT INTO regions(region_name) VALUES
('Moskva'), ('Sankt-Peterburg'), ('Moskovskaya_oblast'),
('Respublika_Tatarstan'), ('Krasnodarskiy_kray');

INSERT INTO periods(period_name, days_count) VALUES

```

```

('Nedelya', 7), ('Mesyats', 30), ('Kvartal', 90), ('God', 365);

INSERT INTO ufps(ufps_name, region_id, address) VALUES
('UFPS_Moskva', 1, 'Moskva, ul. Tverskaya, 1'),
('UFPS_Sankt-Peterburg', 2, 'Sankt-Peterburg, Nevskiy pr., 10'),
('UFPS_Podmoskovye', 3, 'Mytishchi, ul. Mira, 5'),
('UFPS_Tatarstan', 4, 'Kazan, ul. Bauman, 3'),
('UFPS_Kuban', 5, 'Krasnodar, ul. Krasnaya, 21');

INSERT INTO philatelic_products(fp_type_id, product_name, issue_date,
    nominal_value) VALUES
(1, 'Marka_80-letie_Pobedy', '2025-05-01', 80.00),
(1, 'Marka_Kosmos_Rossii', '2025-11-12', 65.00),
(2, 'Otkrytka_Noviy_god_2026', '2025-12-01', 45.00),
(3, 'Konvert_Den_goroda_Moskva', '2025-09-05', 35.00),
(4, 'Blok_Istoriya_aviatsii', '2025-03-18', 120.00),
(5, 'Kartmaksimum_Arktika', '2025-07-22', 90.00);

INSERT INTO requests(ufps_id, request_date, period_id, status) VALUES
(1, '2026-03-02', 1, 'new'),
(2, '2026-03-03', 1, 'new'),
(3, '2026-02-20', 2, 'processed'),
(4, '2025-12-01', 2, 'processed'),
(5, '2026-03-05', 1, 'new'),
(1, '2026-03-05', 1, 'new'),
(2, '2026-01-10', 2, 'processed');

INSERT INTO request_items(request_id, product_id, quantity) VALUES
(1, 1, 120), (1, 3, 80),
(2, 1, 100), (2, 4, 60),
(3, 2, 90), (3, 5, 30),
(4, 1, 40), (4, 6, 15),
(5, 3, 150), (5, 2, 110),
(6, 1, 70), (6, 5, 25),
(7, 2, 50), (7, 3, 20);

```

	fp_type_id [PK] integer	type_name character varying (100)
1	1	Marka
2	2	Otkrytka
3	3	Konvert
4	4	Blok
5	5	Kartmaksimum

Puc. 2

	period_id [PK] integer	period_name character varying (20)	days_count integer
1	1	Nedelya	7
2	2	Mesyats	30
3	3	Kvartal	90
4	4	God	365

Puc. 3

	product_id [PK] integer	fp_type_id integer	product_name character varying (180)	issue_date date	nominal_value numeric (10,2)
1	1	1	Marka 80-letie Pobedy	2025-05-01	80.00
2	2	1	Marka Kosmos Rossii	2025-11-12	65.00
3	3	2	Otkrytka Noviy god 2026	2025-12-01	45.00
4	4	3	Konvert Den goroda Mosk...	2025-09-05	35.00
5	5	4	Blok Istoriya aviatsii	2025-03-18	120.00
6	6	5	Kartmaksimum Arktika	2025-07-22	90.00

Puc. 4

	region_id [PK] integer	region_name character varying (120)
1	1	Moskva
2	2	Sankt-Peterburg
3	3	Moskovskaya oblast
4	4	Respublika Tatarstan
5	5	Krasnodarskiy kray

Рис. 5

	ufps_id [PK] integer	ufps_name character varying (150)	region_id integer	address character varying (255)
1	1	UFPS Moskva	1	Moskva, ul. Tverskaya, 1
2	2	UFPS Sankt-Peterburg	2	Sankt-Peterburg, Nevskiy pr., ...
3	3	UFPS Podmoskovye	3	Mytishchi, ul. Mira, 5
4	4	UFPS Tatarstan	4	Kazan, ul. Baumana, 3
5	5	UFPS Kuban	5	Krasnodar, ul. Krasnaya, 21

Рис. 6

	request_id [PK] integer	ufps_id integer	request_date date	period_id integer	status character varying (30)
1	1	1	2026-03-02	1	new
2	2	2	2026-03-03	1	new
3	3	3	2026-02-20	2	processed
4	4	4	2025-12-01	2	processed
5	5	5	2026-03-05	1	new
6	6	1	2026-03-05	1	new
7	7	2	2026-01-10	2	processed

Рис. 7

	request_item_id [PK] integer 	request_id integer 	product_id integer 	quantity integer 
1	1	1	1	120
2	2	1	3	80
3	3	2	1	100
4	4	2	4	60
5	5	3	2	90
6	6	3	5	30
7	7	4	1	40
8	8	4	6	15
9	9	5	3	150
10	10	5	2	110
11	11	6	1	70
12	12	6	5	25
13	13	7	2	50
14	14	7	3	20

Рис. 8